

Gateway protocollo-agnostico per la comunicazione bidirezionale nell'ambito del progetto Emerge

Relazione Finale di Tirocinio

Alessandro Gardini

`alessandro.gardini7@studio.unibo.it`

Corso di Laurea in Ingegneria e Scienze Informatiche
Università di Bologna – Campus di Cesena

Tutor Didattico: Mirko Viroli

Laboratorio: E (modalità mista)

Periodo: 16/02/2026 - 05/05/2026

10 giugno 2026

Indice

1	Introduzione	3
2	Tecnologie	3
3	Attività	4
3.1	Studio del dominio applicativo e delle tecnologie coinvolte	4
3.1.1	Studio architettuale del progetto	4
3.1.2	Analisi del dominio	5
3.2	Studio architettuale di sistema e di modulo, implementazione	5
3.2.1	Definizione dell'architettura di sistema e studio sui protocolli da supportare	5
3.2.2	Studio architettuale di modulo e realizzazione	6
4	Conclusioni	8

1 Introduzione

Il presente documento costituisce la relazione conclusiva del tirocinio curricolare svolto presso il laboratorio di ricerca Alma Mater Studiorum – Università di Bologna, sede di Cesena, nell’ambito del corso di Laurea in Ingegneria e Scienze Informatiche. Il tirocinio ha visto il mio coinvolgimento all’interno del [Progetto Emerge](#), parte delle attività del gruppo di ricerca sull’aggregate computing.

L’aggregate computing è un paradigma di programmazione che affronta la complessità dei sistemi distribuiti su larga scala, come le reti di sensori, le smart city e gli sciami di robot, astruendo il comportamento dell’intera collettività di dispositivi in un’unica entità logica. Un’applicazione concreta di questo paradigma è stata realizzata dal gruppo di ricerca in occasione della [Notte Europea dei Ricercatori](#), un evento pubblico di divulgazione scientifica. In quell’occasione è stata presentata una dimostrazione interattiva in cui un team di robot mobili a basso costo riusciva a coordinarsi per formare pattern spaziali (cerchio, linea) e a mostrare capacità di auto-riparazione (self-healing) in seguito a perturbazioni esterne, il tutto grazie a un programma aggregativo scritto con il framework [ScaFi](#).

Nella demo originale, la comunicazione tra il server centrale (che esegue la logica aggregativa) e i robot avveniva esclusivamente tramite il protocollo MQTT, scelto per la sua leggerezza e adatto ai robot basati su ESP32. Tuttavia, questa scelta vincolava l’intero sistema a un singolo protocollo, rendendo difficile l’integrazione di nuovi robot che avessero implementazioni diverse.

L’obiettivo principale del mio tirocinio era pertanto quello di progettare e sviluppare un layer di astrazione delle comunicazioni che permettesse di aggiungere allo swarm, robot in grado di parlare con diversi tipi di protocolli. Un modulo estendibile in base alle necessità.

In particolare, il sistema doveva supportare:

- la modifica del protocollo interno senza dover riscrivere il nucleo dell’applicazione aggregativa;
- l’aggiunta di un nuovo tipo di robot che utilizza un protocollo diverso, semplicemente implementando un adattatore specifico;
- la coesistenza, nella stessa demo, di robot che comunicano con protocolli eterogenei.

Le attività, svolte in parte insieme ad un collega, si sono articolate nelle seguenti fasi:

- studio del progetto;
- creazione di una relazione di dominio project wide;
- creazione di una relazione architetturale specifica al progetto;
- sviluppo applicativo.

2 Tecnologie

All’interno del progetto Emerge erano state utilizzate diverse tecnologie per realizzare i diversi moduli. Per la realizzazione della mia parte ho utilizzato *Python*, in particolare

sfruttando la libreria `asyncio` per scrivere codice concorrente single-thread. Come package manager ho utilizzato `uv`.

Le librerie utilizzate:

- `aiocoap`, per realizzare server CoAP;
- `aiomqtt`, per collegarsi al broker interno;
- `fastapi`, per realizzare server sia HTTP che Websocket;
- `pydantic`, per creare payload standard attraverso tutte le comunicazioni.

Per creare i test ho utilizzato `httpx`, `pytest`, `pytest-asyncio`.

Inoltre, è stato utilizzato Docker, poiché il progetto sarà integrato in un contesto più ampio tramite Docker Compose.

3 Attività

3.1 Studio del dominio applicativo e delle tecnologie coinvolte

Prima dello studio diretto del dominio è stata dedicato del tempo allo studio del paradigma dell'aggregate computing e della sua implementazione all'interno del progetto Emerge. Lo studio teorico è stato condotto in modo volutamente introduttivo, sufficiente a comprendere il ruolo del modulo `aggregate-runtime` all'interno del sistema e a orientarsi nelle fasi successive del tirocinio.

3.1.1 Studio architetturale del progetto

In questa fase, io e il mio collega abbiamo condotto un'analisi dell'architettura del progetto Emerge nella sua versione corrente, producendo un documento di review che descrive i quattro moduli principali e le loro interconnessioni.

Il modulo `aggregate-runtime`, scritto in Scala, costituisce il nucleo computativo del sistema. Si articola in due sottomoduli: `core`, che definisce il ciclo di controllo astratto (lettura dell'ambiente, calcolo delle attuazioni, applicazione) parametrizzato sui tipi dominio-specifici `ID`, `Position`, `Info` e `Actuation`; e `demo`, che ne fornisce la realizzazione concreta per lo sciame fisico, collegando `MqttProvider`, `AggregateOrchestrator` e `RobotUpdateMqtt` per formare la pipeline `EnvironmentProvider` → `Orchestrator` → `EnvironmentUpdate`.

Il modulo `aruco-detector`, scritto in Python con `OpenCV` e `paho-mqtt`, si occupa del rilevamento in tempo reale della posizione e dell'orientamento dei robot tramite marker `ArUco`, pubblicando i dati sul broker MQTT al topic `robots/{id}/position`.

Il modulo `neighborhood-system`, anch'esso in Python, calcola dinamicamente il vicinato di ciascun robot a partire dalle posizioni ricevute via MQTT, supportando due modalità: `full` (tutti i robot come vicini) e `radius` (vicini entro un raggio configurabile). Il vicinato aggiornato viene pubblicato sul topic `robots/{id}/neighbors`.

Il modulo `dashboard` fornisce un'interfaccia di visualizzazione e controllo, le cui connessioni con gli altri moduli sono mediate dal broker MQTT.

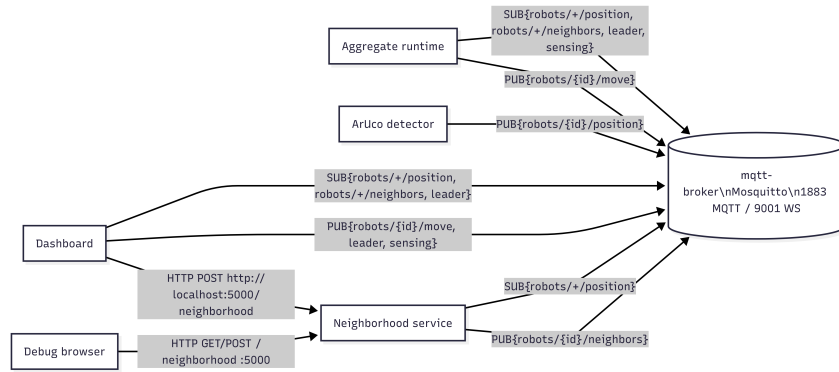


Figura 1: Schema delle connessioni allo stato dell'arte.

L'analisi ha evidenziato come l'intero sistema si appoggi sul broker Mosquitto come unico bus di comunicazione, rendendolo il punto critico attorno a cui ruota qualsiasi estensione multi-protocollo, nell'fig. 1.

3.1.2 Analisi del dominio

In parallelo allo studio architetture, è stata condotta un'analisi del dominio applicativo con l'obiettivo di modellare le entità coinvolte nel sistema in modo agnostico rispetto all'implementazione, senza legarsi a tecnologie o scelte architetture specifiche. Il risultato è un diagramma delle entità che descrive il sistema in un determinato snapshot temporale, è riportato in fig. 2.

3.2 Studio architetture di sistema e di modulo, implementazione

3.2.1 Definizione dell'architettura di sistema e studio sui protocolli da supportare

A partire dall'analisi architetture del sistema esistente, sono state elaborate tre proposte per estendere il progetto Emerge in modo da supportare robot che comunicano con protocolli diversi da MQTT. La prima prevede un server di traduzione diretta, in cui un modulo intermedio si iscrive ai topic del broker per conto dei robot esterni e ne inoltra i messaggi in uscita, preservando il modello ad eventi. La seconda introduce un server che mantiene uno snapshot dell'intero stato del sistema, a cui i robot esterni accedono tramite polling, perdendo però la natura reattiva della comunicazione. La terza propone un redesign più radicale dell'architettura, sostituendo il broker MQTT con un server REST centrale, a vantaggio della manutenibilità ma con un costo di migrazione elevato.

A seguito di un confronto con i tutor, è stato scelto un approccio ibrido tra la prima e la seconda proposta: un modulo di traduzione che, internamente, si interfaccia con il broker MQTT e, verso l'esterno, espone endpoint per protocolli eterogenei, combinando la reattività del modello a eventi con la possibilità di interrogare l'ultimo stato noto di un topic come in fig. 3.

La fase successiva ha riguardato la definizione puntuale di come ciascun protocollo dovesse essere supportato. Per HTTP, è stata prevista una cache che conserva l'ultimo valore pubblicato su ciascun topic: un robot esterno può recuperarlo tramite una richiesta GET e pubblicare verso il broker interno tramite PUT. Per CoAP, la struttura degli endpoint

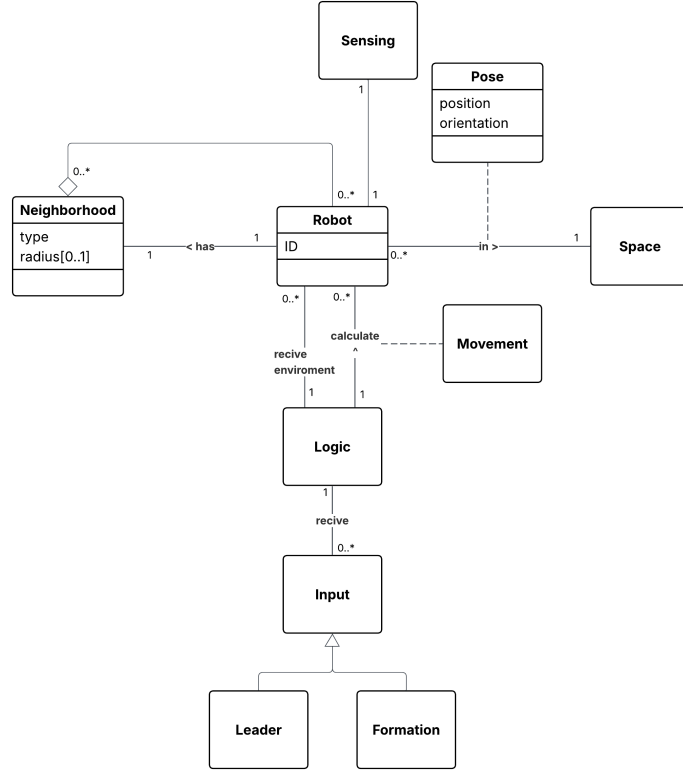


Figura 2: Schema delle entità di dominio.

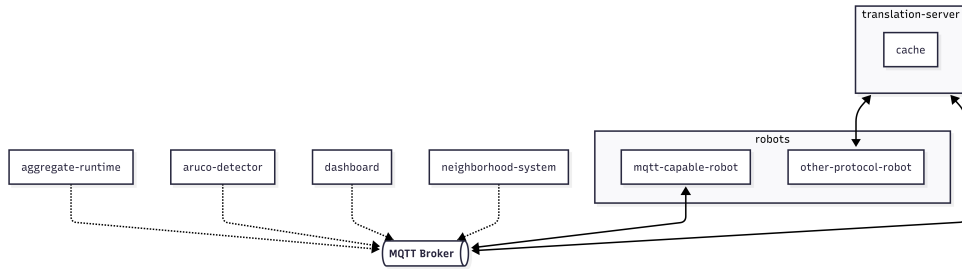


Figura 3: Schema architetturale di sistema scelto.

ricalca quella di HTTP, con l’aggiunta del supporto alla modalità *observe* del protocollo, che consente a un client di ricevere notifiche automatiche al variare del valore di una risorsa, avvicinando il comportamento di CoAP al modello reattivo di MQTT. Per WebSocket, è stato definito un canale bidirezionale persistente: in un verso, i client esterni interessati a un determinato topic ricevono aggiornamenti non appena un nuovo messaggio viene pubblicato sul broker; nell’altro, possono pubblicare messaggi che vengono inoltrati al broker interno.

3.2.2 Studio architetturale di modulo e realizzazione

La fase conclusiva del tirocinio ha riguardato la progettazione architetturale del modulo e la sua realizzazione. Il modulo, denominato **domain-gateway**, si colloca al confine tra il dominio interno del sistema Emerge — interamente basato su MQTT — e i client esterni, che possono comunicare tramite HTTP, WebSocket o CoAP.

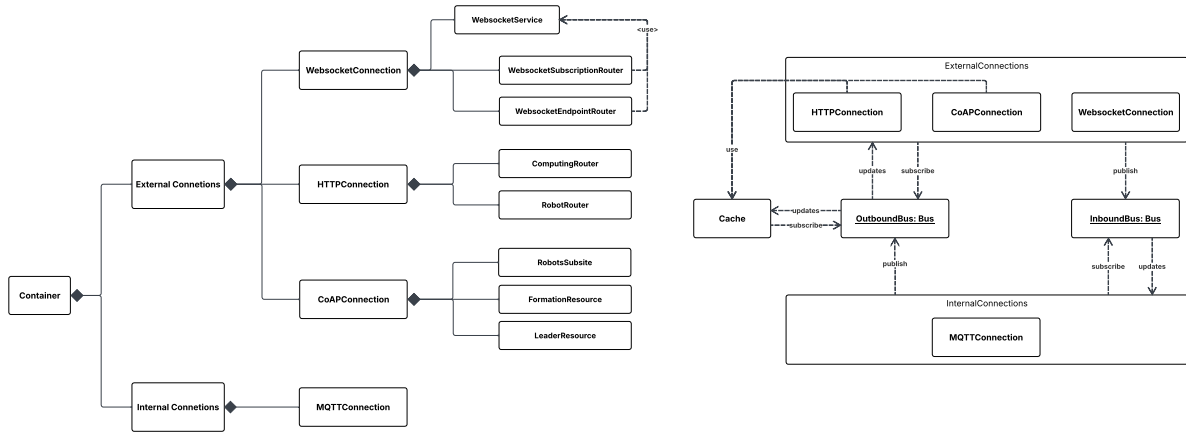


Figura 4: Schemi riassuntivi modulo. Non seguono lo standard UML.

Il principale punto di estensibilità è l’astrazione **Connection**, interfaccia comune a tutti gli adattatori di protocollo. Ogni protocollo — HTTP, WebSocket, CoAP, MQTT e qualsiasi futuro adattatore — implementa la stessa interfaccia, esponendo esclusivamente i metodi `start()` e `stop()`. L’aggiunta di un nuovo protocollo si riduce all’implementazione di questa interfaccia e alla sua registrazione in **ExternalConnections** o **InternalConnections**: nessun’altra parte del sistema necessita di modifiche. La logica di avvio, spegnimento e registrazione dei router è gestita automaticamente dai due gruppi, rendendo `main.py` completamente ignaro dei protocolli specifici in uso.

A supporto di questa estensibilità, la composizione delle dipendenze è affidata a un singolo **Container**, che istanzia e collega tutti i componenti concreti. L’iniezione delle dipendenze via costruttore è applicata in modo uniforme: ogni componente riceve esplicitamente ciò di cui ha bisogno, senza fare riferimento a singleton globali. Sostituire un’implementazione, ad esempio rimpiazzare la cache in memoria con una basata su Redis, è una modifica localizzata a una singola riga nel **Container**.

Il flusso dei messaggi si fonda sulla separazione in due bus distinti: un bus in ingresso (inbound), che trasporta i messaggi dai client esterni verso il broker MQTT, e un bus in uscita (outbound), che distribuisce i messaggi provenienti dal broker verso i client esterni. Tenere i due flussi separati rende la direzionalità del dato esplicita per costruzione. Le operazioni sul bus in ingresso sono gestite con semantica best-effort: i publisher non attendono conferma dai subscriber, mantenendo il bus non bloccante.

Al bus in uscita si sottoscrive anche una cache in memoria, che conserva l’ultimo valore pubblicato per ciascun topic. Questo componente soddisfa la necessità, emersa nella fase di analisi, di permettere ai client esterni di interrogare l’ultimo stato noto del sistema. La cache memorizza un valore solo dopo che il messaggio ha completato il percorso attraverso il broker, garantendo la consistenza dei dati esposti.

Sono stati creati anche due schemi, fig. 4, che cercano di rappresentare l’incapsulamento, risultante dall’utilizzo di **ExternalConnections** e **InternalConnections**, e i flussi dei messaggi all’interno del modulo.

4 Conclusioni

Il tirocinio ha portato alla realizzazione del modulo `domain-gateway`, che estende il progetto `Emerge` con il supporto a protocolli eterogenei, `HTTP`, `WebSocket` e `CoAP`, preservando la compatibilità con l'infrastruttura `MQTT` esistente. L'obiettivo iniziale, rendere il sistema accessibile a robot che non supportano `MQTT`, è stato raggiunto tramite un'architettura ibrida che combina reattività e possibilità di interrogare l'ultimo stato noto del sistema.

Dal punto di vista tecnico, l'esperienza ha permesso di approfondire diversi aspetti rilevanti per la formazione di un ingegnere del software. La progettazione dell'astrazione `Connection` e l'utilizzo sistematico dell'iniezione delle dipendenze hanno reso concreta l'applicazione di principi di design orientati all'estensibilità e alla manutenibilità.

Tra gli aspetti più positivi vorrei menzionare la possibilità di affrontare un problema reale in tutte le sue fasi: dall'analisi del sistema esistente, alla definizione dell'architettura, fino all'implementazione e ai test. Lavorare in un contesto di ricerca ha inoltre favorito un approccio riflessivo alle scelte progettuali, in cui ogni decisione è stata motivata e discussa con i tutor. La collaborazione con un collega nella fase di analisi iniziale ha infine offerto un'occasione di confronto tecnico che ha arricchito entrambe le prospettive.

Complessivamente, il tirocinio ha rappresentato un'esperienza formativa solida, capace di colmare il divario tra le lezioni teoriche viste durante questi anni e l'esperienza reale, e ho potuto consolidare competenze utili in contesti di sviluppo IoT.